

PRD Update: Multi-Source and Multi-Format Data Support for CanonIQ

Add This Section to the CanonIQ PRD

CanonIQ must support different data sources and formats because enterprise schema mapping rarely comes from only CSV files. Users may need to profile and map data from local files, data lakes, data warehouses, cloud storage, SaaS exports, APIs, and databases.

CanonIQ should be designed with a **pluggable connector and profiler architecture** so new source types can be added without rewriting the core mapping engine.

The core design principle is:

CanonIQ should separate data access from schema intelligence. Connectors fetch or sample data. Profilers understand the structure. The mapping engine remains source-agnostic.

1. Supported Data Sources and Formats

MVP v0.1 Required Sources

For the first MVP, implement:

1. Local CSV files
2. Local JSON files
3. Local JSONL files, if simple to add

These are the fastest to build and easiest to demo.

v0.2 Target Sources

Add support for:

1. Local Parquet files
2. Local Excel files
3. Local folders of files
4. SQLite

5. Postgres
6. MySQL

v0.3 Cloud and Enterprise Sources

Add support for:

1. Google BigQuery
2. Snowflake
3. Databricks / Delta Lake
4. Amazon Redshift
5. Microsoft SQL Server
6. Google Cloud Storage
7. Amazon S3
8. Azure Blob Storage
9. Data lake folders and partitioned datasets

v0.4 SaaS and API Sources

Add support for:

1. REST API responses
2. GraphQL responses
3. Salesforce
4. Canvas LMS
5. Shopify
6. HubSpot
7. Workday
8. Stripe
9. Generic SaaS export connectors

2. Source Connector Architecture

CanonIQ should use a connector interface.

Every connector should follow the same contract:

```
from abc import ABC, abstractmethod
from typing import Any, Dict, List
```

```

class BaseSourceConnector(ABC):
    @abstractmethod
    def test_connection(self) -> bool:
        """Verify that the source is reachable."""
        pass

    @abstractmethod
    def list_entities(self) -> List[str]:
        """List available tables, files, collections, or entities."""
        pass

    @abstractmethod
    def sample(self, entity: str, limit: int = 1000) -> List[Dict[str, Any]]:
        """Return a representative sample for profiling."""
        pass

    @abstractmethod
    def get_metadata(self, entity: str) -> Dict[str, Any]:
        """Return source metadata such as table name, schema, path, or
format."""
        pass

```

Rationale

The connector should only handle data access. It should not contain mapping logic. This makes CanonIQ flexible and easy to extend.

For example:

CSVConnector	-> reads local CSV rows
BigQueryConnector	-> samples rows from a BigQuery table
S3Connector	-> reads objects from S3
PostgresConnector	-> samples rows from a database table
SalesforceConnector	-> samples objects from Salesforce

All connectors return data in a common internal format:

```
List[Dict[str, Any]]
```

That allows the profiling and mapping engine to work the same way regardless of source.

3. Profiler Architecture

CanonIQ should also use a profiler interface.

```
from abc import ABC, abstractmethod
from typing import Any, Dict, List

class BaseProfiler(ABC):
    @abstractmethod
    def profile(self, records: List[Dict[str, Any]], source_metadata: Dict[str, Any]) -> Dict[str, Any]:
        """Profile records and return field-level statistics."""
        pass
```

The profiler should calculate:

```
Field names
Inferred data types
Null rates
Unique rates
Sample values
Min/max values
Average string length
Patterns
Enum candidates
Primary-key candidates
PII-like candidates
Timestamp formats
```

Rationale

The profiler should not care whether the data came from CSV, BigQuery, Snowflake, Postgres, S3, or Salesforce. It only needs records and metadata.

4. Source Configuration Format

CanonIQ should support source configuration through YAML.

Example for local CSV:

```
source:
  type: csv
  path: examples/higher_ed/source_students.csv
```

Example for BigQuery:

```
source:
  type: bigquery
  project_id: my-gcp-project
  dataset: student_success
  table: students
  sample_limit: 1000
```

Example for Snowflake:

```
source:
  type: snowflake
  account: my_account
  warehouse: COMPUTE_WH
  database: ANALYTICS
  schema: PUBLIC
  table: CUSTOMERS
  sample_limit: 1000
```

Example for S3 data lake:

```
source:
  type: s3
  bucket: my-data-lake
  prefix: raw/customers/date=2026-05-29/
  format: parquet
  sample_limit: 1000
```

Example for GCS data lake:

```
source:
  type: gcs
  bucket: my-data-lake
  prefix: raw/students/date=2026-05-29/
  format: csv
  sample_limit: 1000
```

Example for Postgres:

```
source:
  type: postgres
  host: localhost
  port: 5432
  database: app_db
  schema: public
  table: customers
  sample_limit: 1000
```

Security Requirement

Credentials should not be stored directly in YAML files. Use environment variables or secrets managers.

Example:

```
source:
  type: postgres
  host: ${POSTGRES_HOST}
  port: ${POSTGRES_PORT}
  database: ${POSTGRES_DB}
  username: ${POSTGRES_USER}
  password: ${POSTGRES_PASSWORD}
  table: customers
```

5. CLI Updates

CanonIQ CLI should support both direct file input and source config input.

File-based profiling

```
canonIQ profile
  --source examples/higher_ed/source_students.csv
  --out profile.json
```

Config-based profiling

```
canoniq profile
  --source-config sources/bigquery_students.yml
  --out profile.json
```

Full mapping workflow

```
canoniq suggest
  --profile profile.json
  --canonical examples/higher_ed/canonical_student.yml
  --out suggestions.json
```

Data lake example

```
canoniq profile
  --source-config sources/s3_product_catalog.yml
  --out product_profile.json
```

BigQuery example

```
canoniq profile
  --source-config sources/bigquery_customer_table.yml
  --out customer_profile.json
```

6. SDK Updates

CanonIQ SDK should support direct files and connectors.

File-based SDK usage

```
from canoniq import CanonIQ

engine = CanonIQ()

profile = engine.profile_csv("examples/retail/source_products.csv")
```

Source-config SDK usage

```
from canoniq import CanonIQ

engine = CanonIQ()

profile = engine.profile_source_config("sources/bigquery_students.yml")
```

Connector-based SDK usage

```
from canoniq import CanonIQ
from canoniq.connectors import BigQueryConnector

engine = CanonIQ()

connector = BigQueryConnector(
    project_id="my-gcp-project",
    dataset="student_success",
    table="students"
)

records = connector.sample(entity="students", limit=1000)
profile = engine.profile_records(records)
```

7. Required Connector Modules

Update the repository structure to include connectors.

```
canoniq/
|
|— connectors/
|   |— __init__.py
|   |— base.py
|   |— csv_connector.py
|   |— json_connector.py
|   |— jsonl_connector.py
|   |— parquet_connector.py
|   |— excel_connector.py
|   |— postgres_connector.py
|   |— mysql_connector.py
|   |— bigquery_connector.py
```



```
| |— snowflake_connector.py
| |— redshift_connector.py
| |— s3_connector.py
| |— gcs_connector.py
| |— azure_blob_connector.py
| |— rest_api_connector.py
```

MVP Implementation Rule

For MVP v0.1, fully implement:

```
csv_connector.py
json_connector.py
jsonl_connector.py
```

For the other connectors, create clean placeholder classes with clear `NotImplementedError` messages and documentation explaining the planned implementation.

Example:

```
class BigQueryConnector(BaseSourceConnector):
    def __init__(self, project_id: str, dataset: str, table: str,
credentials_path: str | None = None):
        self.project_id = project_id
        self.dataset = dataset
        self.table = table
        self.credentials_path = credentials_path

    def test_connection(self) -> bool:
        raise NotImplementedError("BigQueryConnector will be implemented in
v0.3.")

    def list_entities(self) -> list[str]:
        raise NotImplementedError("BigQueryConnector will be implemented in
v0.3.")

    def sample(self, entity: str, limit: int = 1000):
        raise NotImplementedError("BigQueryConnector will be implemented in
v0.3.")

    def get_metadata(self, entity: str):
        raise NotImplementedError("BigQueryConnector will be implemented in
v0.3.")
```

8. Optional Dependency Groups

Do not install all enterprise dependencies by default.

Update `pyproject.toml` with optional dependency groups.

```
[project.optional-dependencies]
files = [
    "openpyxl",
    "pyarrow"
]

databases = [
    "sqlalchemy",
    "psycopg2-binary",
    "pymysql"
]

bigquery = [
    "google-cloud-bigquery"
]

snowflake = [
    "snowflake-connector-python"
]

aws = [
    "boto3",
    "s3fs"
]

azure = [
    "azure-storage-blob"
]

gcp = [
    "google-cloud-storage",
    "google-cloud-bigquery"
]

all = [
    "openpyxl",
    "pyarrow",
    "sqlalchemy",
```

```
"psycpg2-binary",  
"pymysql",  
"google-cloud-bigquery",  
"google-cloud-storage",  
"snowflake-connector-python",  
"boto3",  
"s3fs",  
"azure-storage-blob"  
]
```

Rationale

Open-source users should be able to install the lightweight core package quickly:

```
pip install canoniq
```

Users who need BigQuery can install:

```
pip install "canoniq[bigquery]"
```

Users who need S3 can install:

```
pip install "canoniq[aws]"
```

9. Data Lake Support

CanonIQ should support profiling partitioned data lake paths.

Example S3 path:

```
s3://company-data-lake/raw/orders/date=2026-05-29/hour=10/
```

Example GCS path:

```
gs://company-data-lake/raw/students/date=2026-05-29/
```

Supported formats:

```
CSV
JSON
JSONL
Parquet
Delta Lake, future
Avro, future
ORC, future
```

Data Lake Profiling Strategy

For large data lake folders, do not read everything.

Sampling strategy:

1. List files under path/prefix.
2. Select up to N files.
3. Read up to M rows per file.
4. Combine sample records.
5. Profile sample records.

Default:

```
max_files = 5
rows_per_file = 1000
```

Rationale

Enterprise data lake tables may be very large. CanonIQ should sample intelligently rather than loading full datasets.

10. Data Warehouse Support

CanonIQ should support table sampling from data warehouses.

Target warehouses:

```
BigQuery
Snowflake
Redshift
Databricks SQL
```

Postgres
MySQL
SQL Server

Sampling Strategy

For warehouses, sampling should use SQL query limits.

Example BigQuery:

```
SELECT *  
FROM `project.dataset.table`  
LIMIT 1000
```

Example Snowflake:

```
SELECT *  
FROM DATABASE.SCHEMA.TABLE  
LIMIT 1000;
```

Example Postgres:

```
SELECT *  
FROM public.customers  
LIMIT 1000;
```

Metadata Strategy

When possible, connectors should fetch native schema metadata:

- Column name
- Data type
- Nullable
- Description/comment, if available
- Table name
- Dataset/schema
- Row count estimate, if available

Rationale

Warehouse metadata improves mapping confidence because field types and descriptions may already be available.

11. Enterprise Source Support Priority

Implementation priority:

Phase 1: Local Files

CSV
JSON
JSONL

Phase 2: Rich File Formats

Parquet
Excel
Local folder sampling

Phase 3: Databases

Postgres
MySQL
SQL Server
SQLite

Phase 4: Cloud Warehouses

BigQuery
Snowflake
Redshift
Databricks SQL

Phase 5: Data Lakes

S3
GCS
Azure Blob
Delta Lake

Phase 6: SaaS/API Sources

Salesforce
Canvas
Shopify
Stripe
HubSpot
Workday
Generic REST API

12. Updated Repository Structure

Add the connector package:

```
canoniq/  
|  
├── canoniq/  
│   ├── connectors/  
│   │   ├── __init__.py  
│   │   ├── base.py  
│   │   ├── csv_connector.py  
│   │   ├── json_connector.py  
│   │   ├── jsonl_connector.py  
│   │   ├── parquet_connector.py  
│   │   ├── excel_connector.py  
│   │   ├── postgres_connector.py  
│   │   ├── mysql_connector.py  
│   │   ├── bigquery_connector.py  
│   │   ├── snowflake_connector.py  
│   │   ├── redshift_connector.py  
│   │   ├── s3_connector.py  
│   │   ├── gcs_connector.py  
│   │   ├── azure_blob_connector.py  
│   │   └── rest_api_connector.py
```

```
| |  
| | |  
| | |─ profiler/  
| | |─ matcher/  
| | |─ scoring/  
| | |─ validation/  
| | |─ transform/  
| | |─ drift/  
| | |─ ai/  
| | |─ cli.py  
| |  
| ─ examples/  
| | ─ sources/  
| | | ─ local_csv_students.yml  
| | | ─ local_json_products.yml  
| | | ─ postgres_customers.yml  
| | | ─ bigquery_students.yml  
| | | ─ snowflake_transactions.yml  
| | | ─ s3_orders.yml  
| | | ─ gcs_lms_activity.yml
```

13. Updated Acceptance Criteria

The MVP is complete when these commands work:

```
pip install -e .  
pytest  
canoniq demo higher-ed  
canoniq demo retail  
canoniq demo healthcare  
canoniq demo finance  
canoniq demo logistics
```

And the following source commands work:

```
canoniq profile  
  --source examples/higher_ed/source_students.csv  
  --out profile.json
```



```
canoniq profile
--source-config examples/sources/local_csv_students.yml
--out profile_from_config.json
```

The project should also include placeholder connector classes for:

```
BigQuery
Snowflake
Redshift
Postgres
MySQL
S3
GCS
Azure Blob
REST API
```

Each placeholder must:

1. Follow `BaseSourceConnector` interface
2. Include docstrings
3. Raise clear `NotImplementedError`
4. Mention the planned version for implementation
5. Include comments about required optional dependencies

14. Updated Build Instructions for Claude Code

Update the build plan to include data source support:

1. Build the generic core engine first.
2. Build `BaseSourceConnector`.
3. Build local file connectors:
4. `CSVConnector`
5. `JSONConnector`
6. `JSONLConnector`
7. Build source config loader.
8. Make `canoniq profile` work with:
9. `--source`

10. `--source-config`
 11. Add placeholder enterprise connectors:
 12. BigQueryConnector
 13. SnowflakeConnector
 14. RedshiftConnector
 15. PostgresConnector
 16. MySQLConnector
 17. S3Connector
 18. GCSConnector
 19. AzureBlobConnector
 20. RestAPIConnector
 21. Add optional dependency groups in `pyproject.toml`.
 22. Add docs explaining how to add a new connector.
 23. Add examples for local file source configs.
 24. Keep all enterprise connectors disabled unless optional dependencies are installed.
 25. Ensure no external network calls happen during tests.
 26. Use synthetic data only.
-

15. How This Helps Position CanonIQ

CanonIQ should be described as:

A universal canonical mapping engine that can profile and map data from files, data lakes, databases, warehouses, SaaS platforms, and APIs into trusted canonical models.

This makes CanonIQ valuable beyond one use case. It can support:

AI agent platforms
Customer onboarding
Enterprise SaaS integrations
Healthcare data normalization
Retail product catalog mapping
Finance transaction normalization
Logistics shipment mapping
Data warehouse modernization
Data lake quality workflows

The key message:

CanonIQ is not tied to one domain or one data source. It is a source-agnostic, domain-agnostic canonical mapping engine.